

강민재

Frontend Developer.



데이터 계약과 상태 경계를 설계하는 프론트엔드 엔지니어입니다

모노레포 환경에서 Web/BO/API를 분리하고 Zod 기반 계약 단일화를 통해 **API 타입 중복을 60% 이상 축소**하였습니다.
스키마 변경 시 프론트엔드 **수정 포인트를 약 50% 이상 감소**시키며 컴파일 타임과 런타임 안정성을 동시에 확보하는 구조를 설계했습니다.

복잡한 요구사항을 구조로 정리하며 팀과 기준을 맞춰가는 개발자입니다

검색 파라미터 검증 파이프라인과 캐시 무효화 전략을 설계하며 기능 구현을 넘어 책임과 경계를 명확히 정의하는 방식으로 문제를 해결했습니다.
기술 선택의 이유를 설명하고, 팀이 이해 가능한 구조로 정리하는 것을 중요하게 생각합니다.

About.

Phone 010.3381.5862

Email minijae011030@gmail.com

Tech Blog <https://blog.minjae-dev.com>

GitHub <https://github.com/minijae011030>

Experience.

- **Megazone Cloud** · Frontend Intern · 2025.11 - 2026.01
 - 사내 솔루션 백오피스 마이그레이션
 - TanStack Router 기반 URL 상태 검증 파이프라인 설계
 - TipTap 에디터 렌더링 병목 개선

Awards.

- 세종대학교 **SFM 외식서비스 앱 기획 및 개발 경진대회** · 우수상 (1등)
 - 주문 공유 서비스 기획 및 MVP 구현
- **KB IT's Your Life 해커톤** · 장려상 (4등)
 - Vue.js + TypeScript 기반 프론트엔드 개발 담당

Skills.

- HTML5 · CSS3 · JavaScript · TypeScript · TailwindCSS
- React · Next.js (App Router) · TanStack Router · Tanstack Query · Zustand
- NestJS · Spring Boot · Zod · pnpm Monorepo

Certificates.

- **SQLD** · 한국데이터산업진흥원 · 2026.03



메가존클라우드 인턴

2025.11 - 2026.01

AS-IS

- React Router 기반 페이지 단위 URL 상태 처리
- 검색 파라미터 파싱/검증 로직 화면별 분산
- JavaScript 기반 타입 안정성 부재
- 이탈 방지 로직 화면별 개별 구현

TO-BE

- TanStack Router 기반 URL 상태 검증/정규화 중앙화
- validateSearch 기반 URL 검증/정규화 파이프라인 설계
- TypeScript 전환 및 타입 안정성 확보
- 공통 UnsavedChangesGuard 도입으로 이탈 정책 중앙화

사내 솔루션 백오피스 프론트엔드 마이그레이션 프로젝트

Tech Stack

- React · TypeScript · Tanstack Query · Tanstack Router

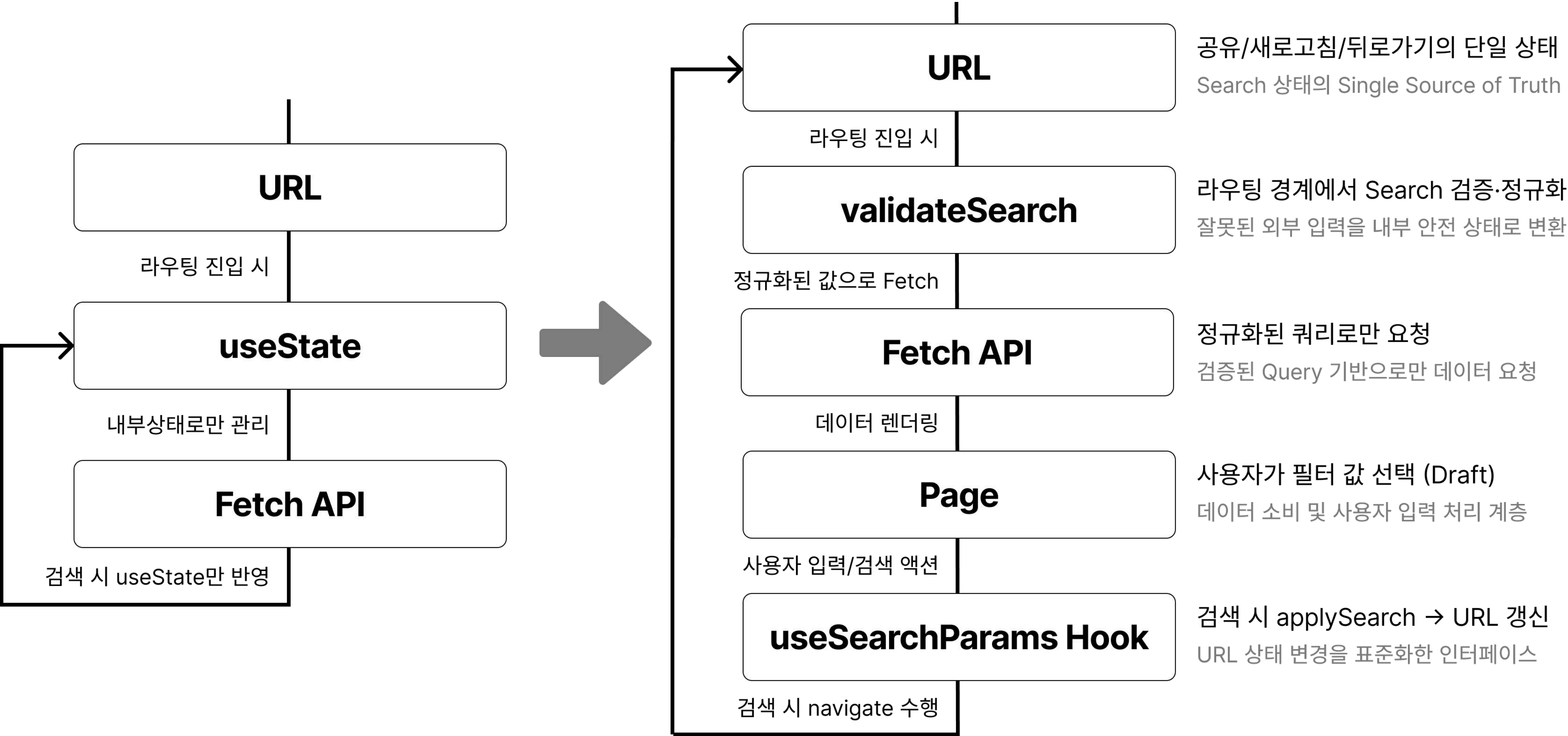
What I Designed

- JavaScript → TypeScript 전환
- TanStack Router 기반 URL 검증 파이프라인
- 공통 UnsavedChangesGuard 컴포넌트 설계
- 에디터 렌더링 경로 구조 개선

Structural Impact

- useSearchParams 구조 41개 페이지 확산 적용
- URL 파싱/정규화 로직 중앙화
- 이탈 방지 정책 5개 화면 공통화 → 변경 포인트 1곳
- 편집 경로 base64 제거 → 렌더링 경로 경량화

TanStack Router 기반 타입 안전 Search 관리 구조 설계



[구조적 한계] URL 미반영 검색 상태 관리

- 검색 상태를 **useState**로만 관리하여 URL에 반영되지 않는 구조
- 새로고침 / 뒤로가기 / 공유 링크 진입 시 상태가 초기화되어 동일 화면 재현 불가
- 상태 관리 로직이 페이지 내부에 분산되어 **일관성 없는 구현** 발생
- 브라우저 히스토리와 UI 상태가 불일치하는 구조적 한계

[설계 전략] Routing Boundary 재설계

- 검색 상태를 컴포넌트 내부가 아닌 **URL 기반 상태로 재정의**
- TanStack Router의 validateSearch 단계에 **Zod 기반 검증/정규화 파이프라인 구성**
- **URL 상태 변경을 표준화**하기 위해 useSearchParams 설계
 - applySearch(patch) / resetSearch() 인터페이스 제공
 - Draft(입력)와 Commit(URL 반영) 상태 분리

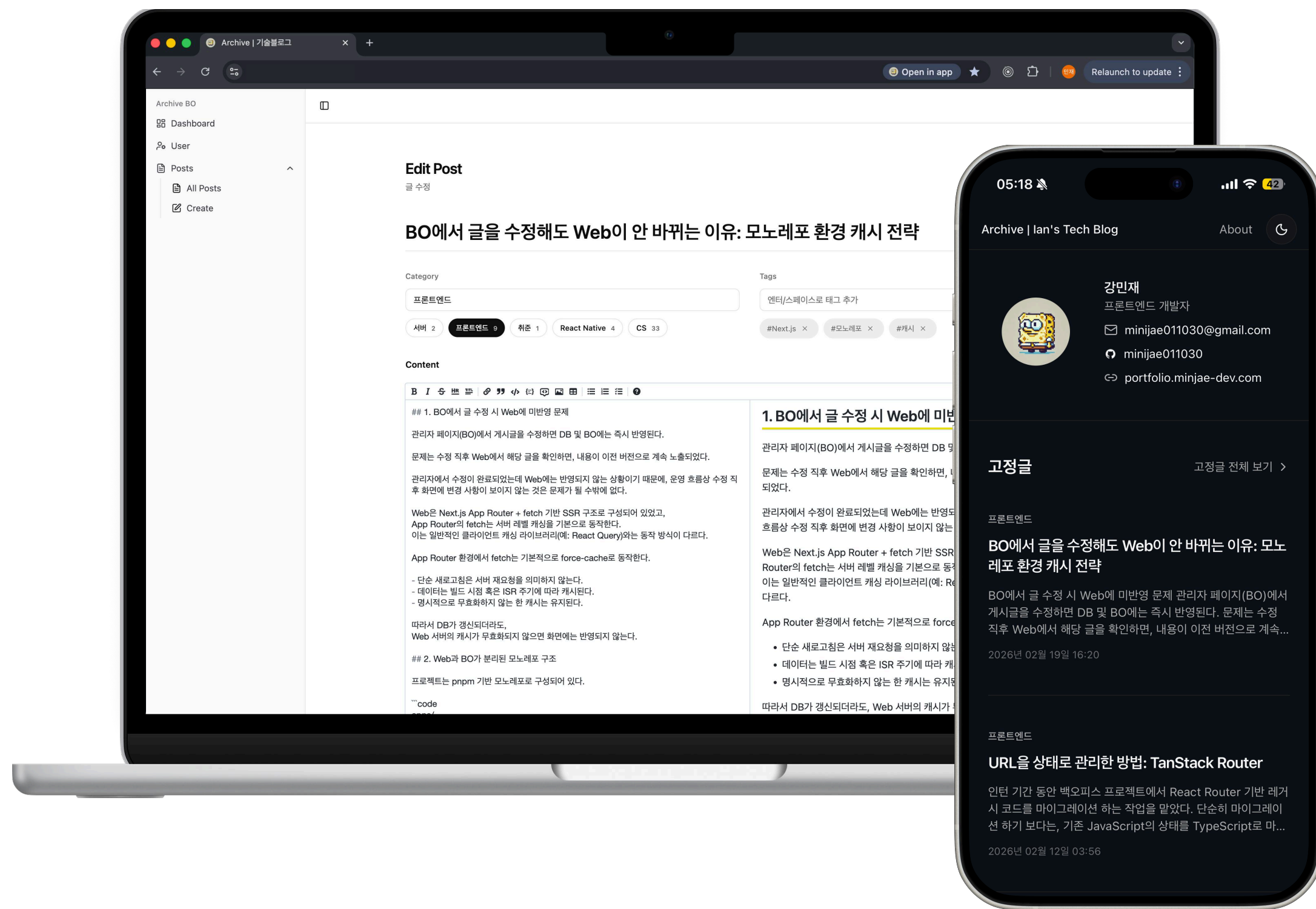
[개선 효과]

- URL에 임의 값 입력 시에도 validateSearch 단계에서 정규화되어 **잘못된 서버 요청 차단**
- 검색 상태 변경 로직을 공통 훅으로 추상화하여 페이지별 **중복 코드 약 40% 감소**
- 공유/새로고침/뒤로가기 상황에서도 URL을 **Single Source of Truth**로 유지하여 상태 일관성 확보

Tech Blog Platform

[Site](#)[GitHub](#)

2024.06 - 2026.03



Web/BO/API 분리 아키텍처 기반 개인 블로그 플랫폼

Tech Stack

▸ Next.js(App Router) · NestJS · Monorepo(pnpm + Turborepo) · Zod

What I Designed

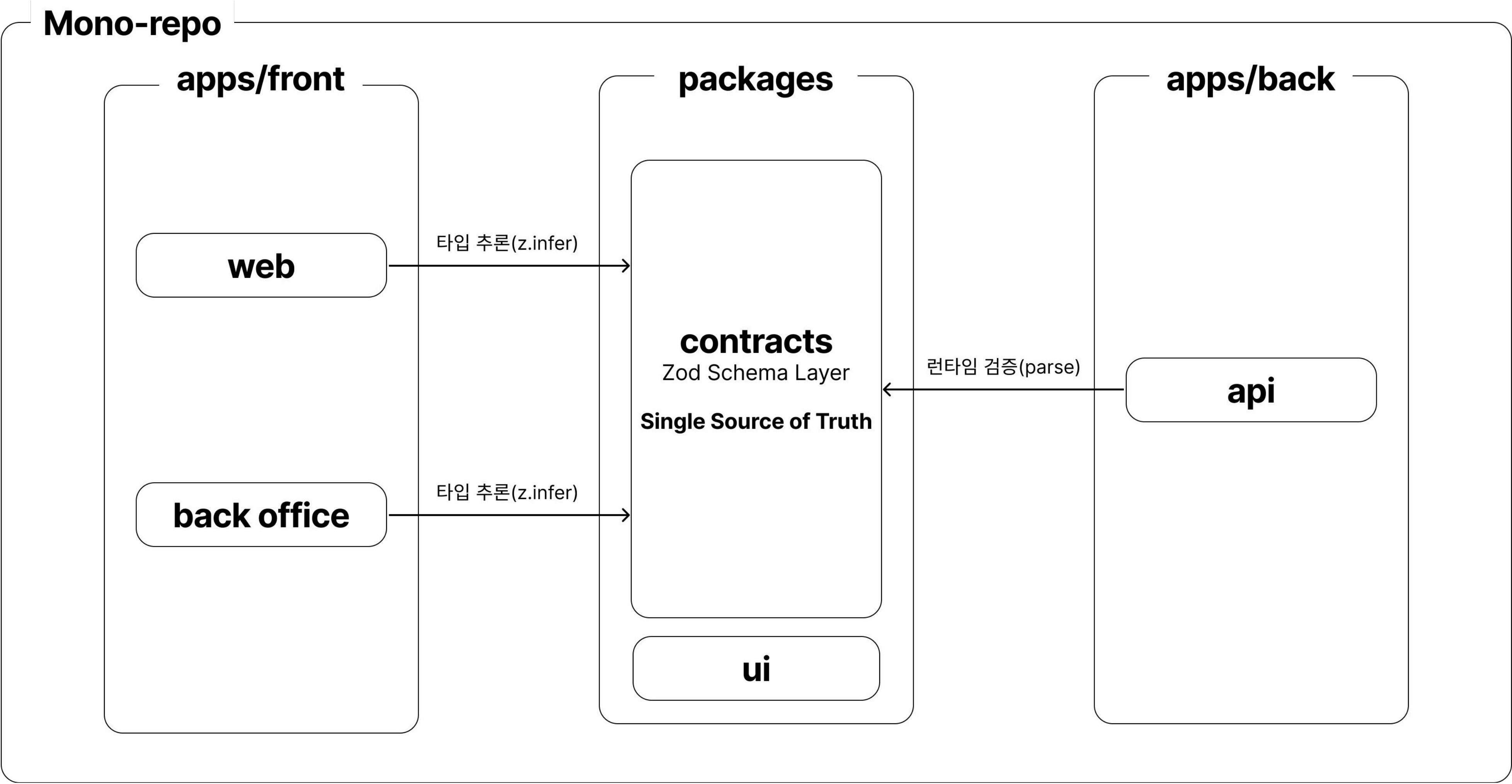
- Web / BO / API 애플리케이션 분리 구조 설계
- pnpm Workspace + Turborepo 기반 Monorepo 빌드 파이프라인 설계
- Zod 기반 API Contract 단일화
- Next.js Tag 기반 캐시 무효화 전략 설계

Structural Impact

- API 타입 정의 중복 약 60~70% 축소
- 스키마 변경 시 프론트 수정 포인트 약 50% 이상 감소
- Web/BO/API 독립 배포 구조 확립
- 태그 기반 선택적 캐시 무효화로 전체 purge 없이 즉시 반영

Monorepo 기반 Contract-First 아키텍처 설계

Web / BO / API 경계 분리 및 Schema 중앙화



[구조적 한계] 물리적 분리 vs 논리적 분산의 문제

- Web / BO / API가 물리적으로 분리되었으나 데이터 계약은 여전히 각 애플리케이션에 분산 관리
- 타입 중복 선언 및 스펙 불일치 리스크 존재
- API 변경 시 변경 전파 비용 증가

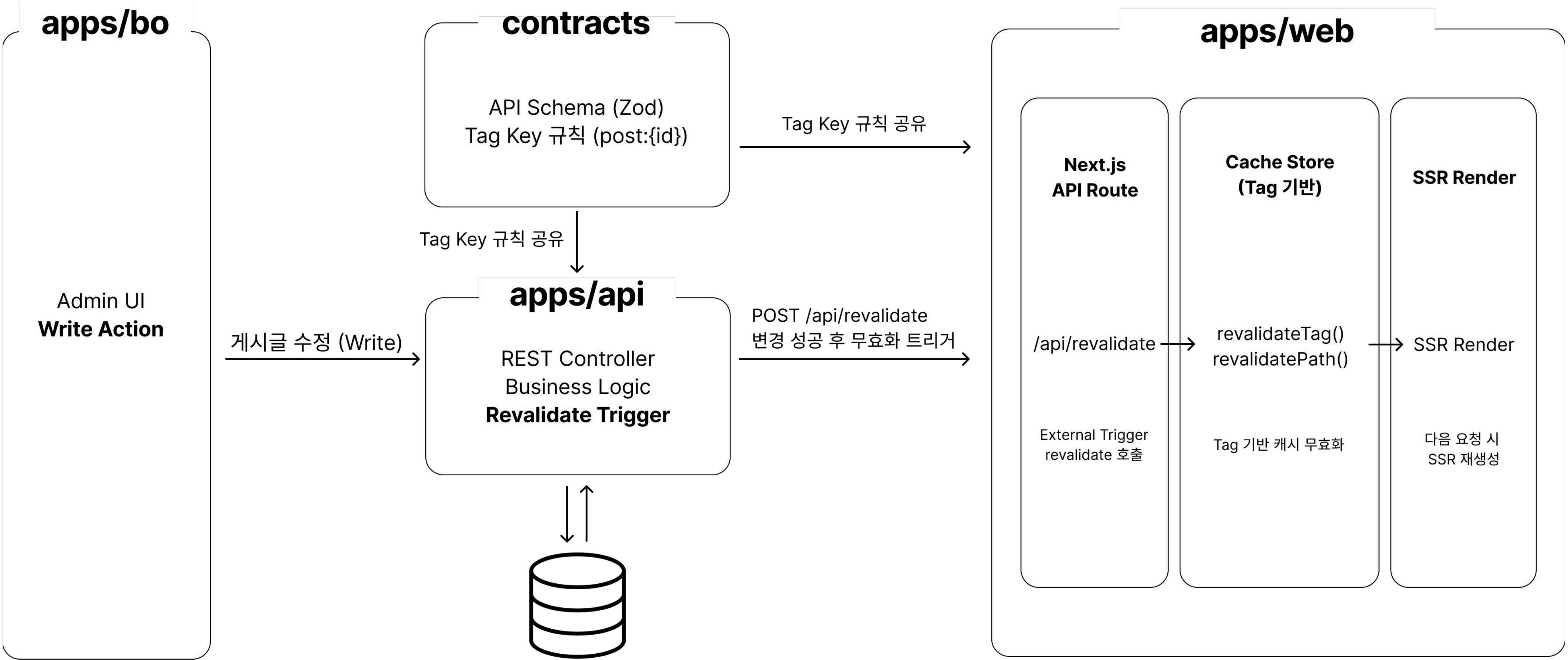
[설계 전략] Monorepo 기반 Schema Layer 중앙화 전략

- pnpm Monorepo 구조 내 packages/contracts를 애플리케이션 간 계약 계층으로 명확히 분리
- Zod 기반 Schema Layer를 **Single Source of Truth**로 정의
- Web / BO → z.infer 기반 컴파일 타입 추론, API → schema.parse 기반 런타임 검증
- Node / Web 환경 호환을 위해 contracts 패키지를 **CJS / ESM dual build** 구조로 구성

[개선 효과]

- 타입 정의 지점 3 → 1로 수렴해 **타입 관리 포인트 약 66% 감소**
- API 변경 시 contracts 패키지 수정만으로 전 애플리케이션에 반영되는 구조 확보
- 컴파일 타임 + 런타임 이중 안전장치로 엔드투엔드 타입 안정성 강화
- 물리적 애플리케이션 분리와 논리적 데이터 계약 일관성을 동시에 확보

Web 캐시 소유권 기반 선택적 무효화 파이프라인



[구조적 한계] 런타임 분리 환경에서의 캐시 불일치

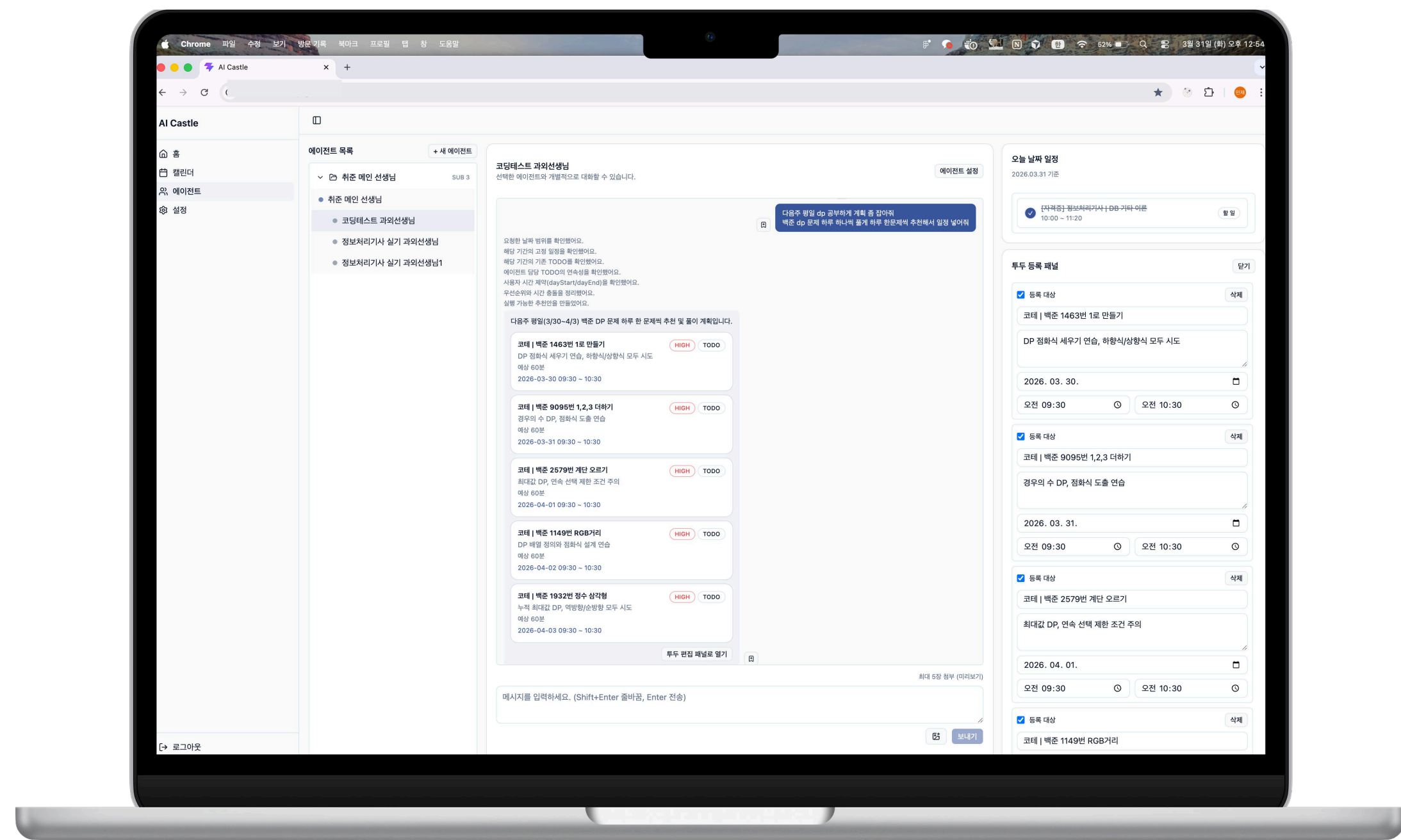
- BO에서 게시글 수정 후 DB에는 반영되지만 **Web에는 즉시 반영되지 않음**
- Web은 App Router 기본 **force-cache 전략**으로 데이터 캐시 유지
- revalidateTag / revalidatePath는 Web 서버 내부에서만 유효
- **전체 no-store 적용 시 서버 비용 증가 및 성능 저하 발생**

[설계 전략] Web 캐시 소유권 기반 Tag 무효화 파이프라인

- 캐시 소유권을 Web으로 명확히 정의하고, 무효화는 Web 내부 API Route를 통해서만 수행
- 도메인 + ID 단위 Tag 체계 설계 (post, post:{id})
- API 변경 성공 시 **Web의 /api/revalidate 호출** 파이프라인 구성
- 무효화 규칙을 **contracts 계층에서 정의하여 정책 중앙화**

[개선 효과]

- BackOffice 수정 → Web에 즉시 반영되는 구조 확보
- 전체 purge 없이 변경 범위만 선택적 갱신
- **no-store 없이 성능과 최신성 동시 확보**
- 캐시 무효화 책임을 명확히 분리하여 구조적 안정성 강화



계층형 멀티 에이전트 기반 주도형 학습 일정 관리 시스템

Tech Stack

▸ React · Spring Boot · OpenAI API

What I Designed

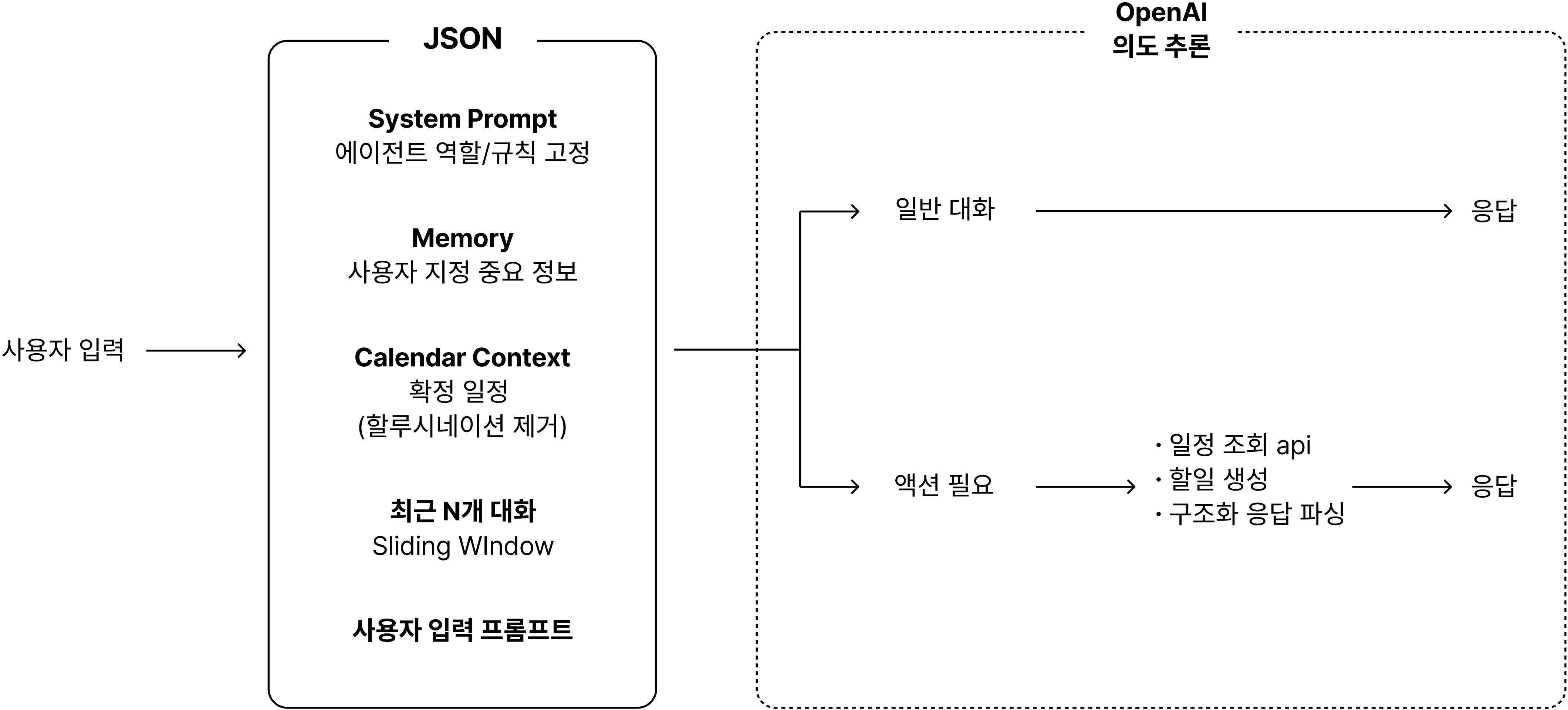
- 상위/하위 에이전트 계층 구조 설계 (Supervisor Orchestration)
- 사용자 지정 업무 시작·종료 시간 기반 자동 배치로 메인 → 서브 에이전트 순차 실행
- 사용자 캘린더 일정 RAG 방식 주입으로 할루시네이션 방지
- 자연어 입력 기반 의도 분류 구조 설계

Structural Impact

- 일정 관리 책임이 사용자 → 시스템으로 이전되는 주도형 구조 확보
- 컨텍스트 길이와 무관하게 에이전트 일관성 유지 가능한 파이프라인 구축
- 사용자 입력 없이도 업무 시작/종료 시 자동으로 할 일 생성 및 리포트 발행
- 액션과 대화를 분기 처리해 단일 인터페이스로 복합 기능 수행 가능

컨텍스트 제어 기반 AI 에이전트 응답 흐름

JSON 구조화 요청 및 의도 추론 기반 분기 처리



[구조적 한계] 단일 대화창 기반 AI 일정 관리의 한계

- 컨텍스트가 누적될수록 **AI가 앞선 내용을 부정확하게 요약**하며 응답 품질 저하
- 사용자가 확정한 일정이 없어 AI가 일정을 추측하거나 존재하지 않는 계획 생성
- **단일 에이전트 구조**로 전체 일정 조율과 과목별 세부 관리를 동시에 수행 불가

[설계 전략] 계층형 멀티 에이전트 + 컨텍스트 제어 구조

- 상위 에이전트(전체 조율) / 하위 에이전트(과목별 할 일 생성) 계층 분리
- 에이전트가 호출 가능한 일정 조회 API를 명령어로 정의해 AI 추측 기반 할루시네이션 방지
- 사용자가 직접 중요 정보를 메모리에 추가할 수 있는 구조로 **핵심 컨텍스트 유실 방지**
- 사용자 입력 프롬프트 + 최근 N개 대화만 전송하는 방식으로 토큰 비용과 맥락 품질 동시 제어

[개선 효과]

- 에이전트가 일정 API를 직접 조회해 **추측 없이 정확한 일정 기반 응답 생성**
- 사용자 주도 메모리 관리로 중요 정보 유실 없이 대화 지속 가능
- 최근 N개 대화 기반 컨텍스트 전송으로 토큰 비용 절감 및 응답 품질 유지
- **역할 분리된 계층 구조로 에이전트별 페르소나 일관성 확보**